

1 Mapa e ramificação técnica da Samsung Xclipse 940

Projeto: XO940

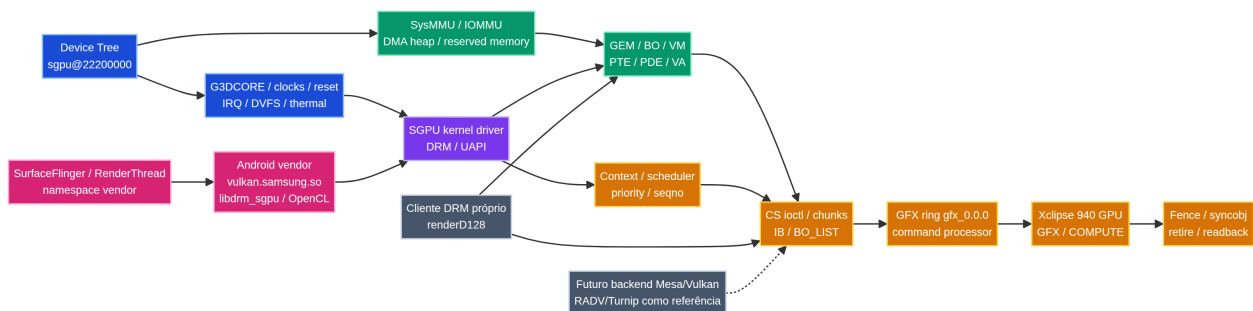
Hardware: Samsung Xclipse 940 — SM-S721B / s5e9945

Finalidade: representação visual da infraestrutura, dos caminhos de integração e das principais falhas de portabilidade

1.1 Legenda cromática

Cor	Significado
Azul	Plataforma e recursos descritos pela Device Tree: power-domain, clocks, reset, IRQ, DVFS, thermal e relações do SoC.
Roxo	Kernel SGPU/DRM e interfaces de baixo nível.
Verde	Memória, GEM/BO, VM, VA, PTE/PDE, IOMMU e DMA.
Laranja	Command submission, contextos, scheduler, IBs, rings, GPU e sincronização.
Rosa	Componentes Android vendor, loader, SurfaceFlinger, RenderThread e namespaces.
Cinza	Cliente próprio, integração experimental ou futuro backend Mesa/Vulkan.
Verde escuro no mapa de falhas	Caminho funcional desejado, ainda dependente de validação fim a fim.
Amarelo no mapa de falhas	Camada parcialmente compreendida ou que exige correlação adicional.
Vermelho no mapa de falhas	Falha ou brecha técnica possível que ainda pode interromper a cadeia.
Azul escuro no mapa de falhas	Fronteira externa ou dependência de integração, como Android, shader backend ou WSI.

1.2 Mapa visual da infraestrutura



1.3 Ramificação técnica

SoC Exynos / Android

- └─ Xclipse 940 / bloco G3D
 - └─ Device Tree
 - | └─ sgpu@22200000
 - | └─ pd_g3dcore
 - | └─ MMIO: gpu, doorbell, debug, pwrctl, sysreg, htu
 - | └─ IRQ: SGPU e GPU-AFM
 - | └─ clocks, DVFS, IFPO, thermal e reset
 - | └─ memória reservada, DMA heap, BTS e SysMMU/IOMMU
 - └─ Kernel SGPU / DRM
 - | └─ sgpu_drm.h / UAPI
 - | └─ GEM, BO, TTM e DMA-BUF
 - | └─ VM, PTE, PDE, VA e flush
 - | └─ context, scheduler, prioridade e seqno
 - | └─ CS, chunks, IB e BO_LIST
 - | └─ GFX/COMPUTE, rings e command processor
 - | └─ fences, syncobjs, dependências e retire
 - | └─ firmware, timeout, reset e recovery
 - └─ Android vendor
 - | └─ libdrm_sgpu.so
 - | └─ vulkan.samsung.so
 - | └─ libOpenCL.so / libSGPUOpenCL.so
 - | └─ SurfaceFlinger / RenderThread / Gralloc
 - | └─ linker namespaces, SELinux e HAL
 - └─ Cliente independente
 - └─ /dev/dri/renderD128
 - └─ BO/VA/BO_LIST/context já acessíveis
 - └─ uma entrada CS já aceita pelo KMD
 - └─ execução GPU e readback ainda não demonstrados
 - └─ backend Vulkan/Mesa experimental ainda não implementado

1.4 Caminho operacional

Device Tree / power / clocks / reset

↓

SGPU kernel driver / DRM UAPI

↓

GEM/BO + VM/PTE/PDE + IOMMU

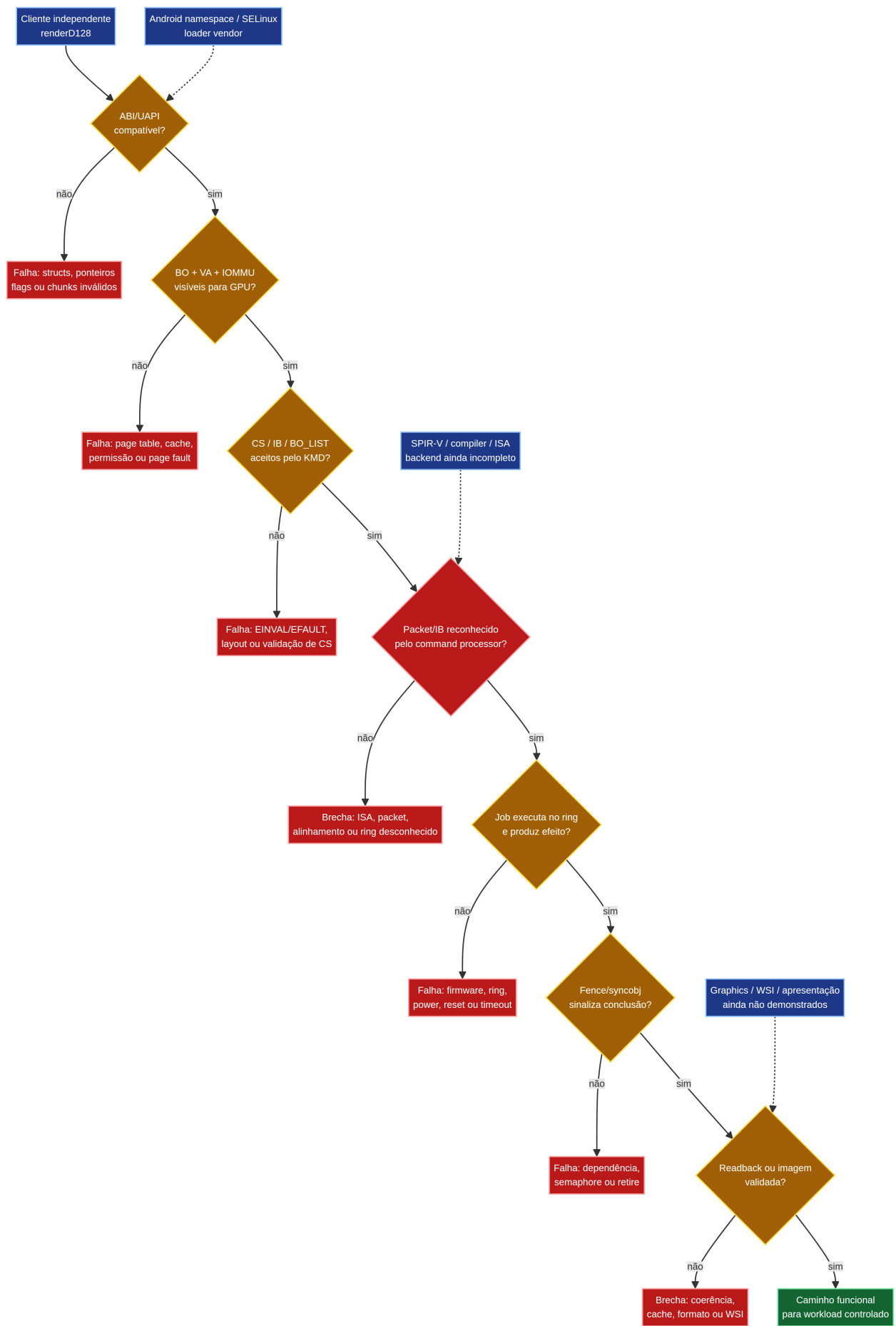
↓

context + BO_LIST + CS chunks + IB

```
↓  
scheduler + ring gfx_0.0.0 + command processor  
↓  
GPU GFX/COMPUTE  
↓  
fence / syncobj / retire  
↓  
readback ou apresentação
```

No caminho de produção, SurfaceFlinger e outros processos Android vendor já atravessam a pilha proprietária. No caminho de estudo, o cliente próprio já acessa renderD128, prepara BO/VA/BO_LIST/ contexto e obteve uma aceitação específica de CS. A etapa de execução GPU validada ainda não está fechada.

1.5 Mapa de possíveis falhas e brechas



O segundo mapa organiza as camadas que podem impedir uma implementação independente:

Camada	Falha ou brecha possível	Sintoma esperado
ABI/UAPI	Estrutura, ponteiro, flag ou chunk incompatível	EINVAL, EFAULT ou ioctl rejeitado
Memória	BO não visível, VA incorreto, cache ou IOMMU	page fault, ausência de efeito ou timeout
CS/IB	BO_LIST, VA, tamanho, alinhamento ou ip_type incorreto	CS aceito parcialmente ou rejeitado pelo KMD
Packet/ISA	command processor não reconhece o conteúdo do IB	job sem efeito, fault ou reset
Ring/firmware	ring, doorbell, firmware, power ou reset não coerentes	timeout, hang ou ausência de retire
Sincronização	dependência, fence, semaphore ou syncobj incorreto	job pendente ou conclusão não observável
Readback	coerência, formato, cache ou recurso de saída incorreto	resultado indefinido mesmo com submit aceito
Android	namespace, SELinux, HAL ou loader vendor	ICD invisível ou bibliotecas não carregadas
Shader backend	SPIR-V, IR, encoder e ISA ainda não correlacionados	pipeline sem execução ou código incompatível
Graphics/WSI	render target, apresentação e surface não implementados	ausência de imagem ou swapchain funcional

1.6 Interpretação

Uma seta no mapa representa uma relação técnica documentada ou uma dependência de implementação. Ela não significa que cada caminho já foram executados por um cliente independente. Os ramos de shader, graphics, WSI e backend Mesa continuam em fase de investigação.

1.7 Referências

1 2 3